

PBA: 3. Data Types/Structures & Data Visualization

Fall 2025

Jaewon (“Jay-one”) Yoo

National Tsing Hua University

Roadmap

1. **Data types and structures I:**

- Numeric, character/string, factor, logical, date/time
- Vectors, matrices, and arrays

2. **[First glance at] visualization for exploratory analysis:**

- Building plots by layers
- Geoms (i.e., geometric objects) are functions!
- Adding informative labels
- Mapping versus setting aesthetics

Numeric and Integers in R

- **Two Main Types**

- **Numeric** (with decimal; also known as double)
- **Integer** (whole number)

- **Default Behavior**

- Numeric values stored in a variable are automatically assumed to be of type numeric.
- Append the value with an 'L' to specify as integer (e.g., 5L).

- **Examples:**

```
1 > i <- 5L
2 > is.integer(i)
3 > class(i) # or use typeof() or mode()
4 > class(4L * 2.8)
5 > class(5L / 2L)
```

Characters/Strings in R

- **What is it?**
 - Sequences of texts enclosed in single (') or double quotes (").
 - Case-sensitive: "Data" ≠ "data" ≠ "DATA".
- **Manipulating Characters**
 - Concatenation: `paste()`, `paste0()`.
 - Substring: `substr()`.
 - String length: `nchar()`.
- **Examples:**

```
1 > x <- "data"
2 > class(x)
3 > nchar(x)
4 > paste("Hello", x)
5 > substr(x, 1, 2)
```

Factors in R

- **Factor Fundamentals**

- Ideal for categorical data (e.g., gender).
- Internally stored as integers, displayed as **character** labels.

- **Why Use Factors?**

- Ensure correct treatment of categorical variables in statistical models.
- Efficient storage and data manipulation.

- **Common Functions**

- Get or set factor levels: **levels()**.
- Convert to factor: **as.factor()**.
- Create a factor: **factor()**.

- **Examples:**

```
1 > gender <- factor(c("Male", "Female", "Male"))
2 > levels(gender)
3 > as.integer(gender)
4 > ordered_gender <- factor(gender, ordered = TRUE)
```

Logical/Booleans in R

- **Logicals (Booleans):** Derived from Boolean algebra by George Boole, represent data as **TRUE** or **FALSE**.
 - Result from comparisons of numbers or characters.
 - Numerically, non-zero is **TRUE**, zero is **FALSE**.
- **Examples:**

```
1 > TRUE * 5      # Output: 5
2 > FALSE * 5     # Output: 0
3 > 2 == 3        # Output: FALSE
4 > 2 != 3        # Output: TRUE
5 > 2 < 3         # Output: TRUE
6 > 2 >= 3        # Output: FALSE
7 > "data" == "stats" # Output: FALSE
8 > "data" < "stats"  # Output: TRUE
```

More on Logical Operations

- & (AND, Conjunction $\wedge$): TRUE if both are TRUE.
- | (OR, Disjunction $\vee$): TRUE if at least one is TRUE.
- ! (NOT, Negation $\neg$): Reverses the logical value.
- **Examples:**

```
1 > 3 & 0 ## FALSE
2 > 3 | 0 ## TRUE
3 > !0    ## TRUE
4 > 3 & 1 ## TRUE
5 > 3 | 2 ## TRUE
6 > !1    ## FALSE
```

Logical Operations in Practice

- Customer Segmentation
 - Logical operations can be used to define business rules.
 - Example: classifying customers as VIP or AtRisk.
- **Examples:**

```
1 > customers <- data.frame(  
2   id = 1:6,  
3   spend = c(500, 1200, 300, 750, 2000, 150),  
4   visits = c(12, 4, 30, 7, 20, 1)  
5 )  
6  
7 # Define "VIP" customers: high spend AND frequent visits  
8 > customers$VIP <- (customers$spend > 1000) &  
9   (customers$visits > 10)  
10  
11 # Define "at-risk": low spend OR very few visits  
12 > customers$AtRisk <- (customers$spend < 400) |  
13   (customers$visits < 5)  
14  
15 > customers  
16   id spend visits  VIP AtRisk  
17 1  500    12 FALSE  FALSE  
18 2 1200     4  FALSE   TRUE  
19 3  300    30  FALSE   TRUE  
20 4  750     7  FALSE  FALSE  
21 5 2000    20   TRUE  FALSE  
22 6  150     1  FALSE   TRUE
```


Another Practical Use Case

- Data Cleaning
 - Logical checks are powerful for identifying invalid or missing data.
 - Example: flagging rows with impossible ages or missing income.
- **Examples:**

```
1 require(tidyverse)
2 > df <- tibble(
3   age = c(25,-3,45,200,30),
4   income = c(50000,20000,NA,100000,30000)
5 )
6
7 > invalid <- (df$age < 0) | (df$age > 120) | is.na(df$income); invalid
8 [1] FALSE TRUE TRUE TRUE FALSE
9
10 > df[invalid, ]
11
12 # A tibble: 3 × 2
13   age income
14   <dbl> <dbl>
15 1    -3  20000
16 2    45    NA
17 3   200 100000
```

Dates and Time in R

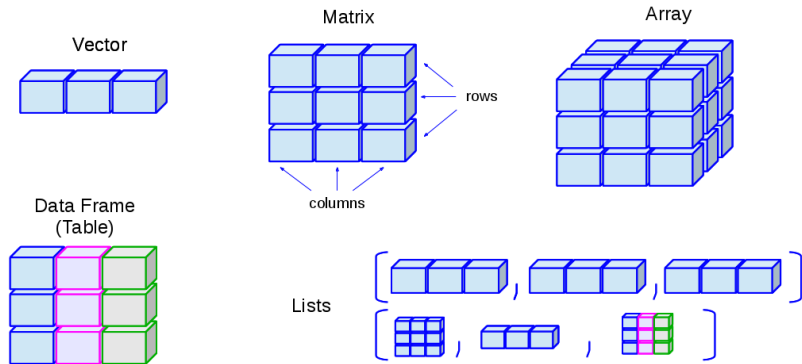
- R manages dates with **Date**, and date-time with **POSIXct** or **POSIXlt**.
 - Stored as days (Date) or seconds (POSIXct) since 1970-01-01 (Epoch).
 - POSIXlt is stored as a **list** of components (year, month, day, hour, minute, second, etc.).
- **Examples:**

```
1 > date1 <- as.Date("2012-06-28")
2 > class(date1)
3 [1] "Date"
4 > as.numeric(date1)
5 [1] 15519
6
7 > date2 <- as.POSIXlt("2012-06-28 17:42")
8 > class(date2)
9 [1] "POSIXlt" "POSIXt"
10
11 > as.data.frame(unclass(date2))
12   sec min hour mday mon year wday yday isdst zone gmtoff
13 1    0  42   17   28    5  112    4  179     0  KST      NA
14
15 > Sys.Date() - date1 # Sys.time() for system date-time!
16 Time difference of 4829 days
```

- For easier date/time manipulation, we'll use **lubridate** package.
 - ~ More on this later!

R provides data structures as a flexible foundation to work with data.

Data Structures



- Vectors (1D), matrices (2D), arrays (nD) can store only **single data type** whereas lists (1D) and data frames (2D) can store **multiple data types**.

~> nD = n Dimensional.

Vectors: Simplest Form of Data Structure

Generating Vectors in R

- Generating vectors is a fundamental step in data manipulation in R.
- Various functions and operators are available for this purpose:
 1. `c()`: combine or concatenate values
 2. Colon operator (`:`) or `seq()`: generate regular sequences
 3. `rep()`: replicates the values in x
- **Examples:**

```
1 > x <- c(1,2,4)
2 > 5:8
3 > seq(5,8)
4 > rep(1:3,2)
5 > i <- 2 # Generate vector of length 1
6
7 > 1:(i-1)
8 > 1:i-1 # Why is the output different? precedence!
```

Arithmetic Operations on Vectors

- Operations are applied to vectors **element-wise**!

- Basic Arithmetic Operations:

```
1 > x <- c(1,2,4); y <- c(5,0,-1) # Generate vectors x and y
2 > x + y # Addition: 6 2 3
3 > x - y # Subtraction: -4 2 5
4 > x * y # Multiplication: 5 0 -4
5 > x / y # Division: 0.2 Inf -4
6 > x^y # Exponentiation: 1.00 1.00 0.25
```

- Modular Operations:

```
1 > x %% y # Modulus (x mod y): 1 NaN 0
2 > x %/% y # Integer division: 0 Inf -4
```

- **Matrix Operations:**

```
1 > x %*% y # Matrix multiplication: 1 (as matrix)
```

↪ Matrices are essentially multidimensional vectors!

Logical Operations on Vectors

- **Element-wise comparison:**

```
1 > x <- c(1,2,4); y <- c(5,0,-1) # Generate vectors x and y
2
3 x > y # Greater than: FALSE TRUE TRUE
4 x <= y # Less than or equal to: TRUE FALSE FALSE
5 x == y # Equal to: FALSE FALSE FALSE
6 x != y # Not equal to: TRUE TRUE TRUE
```

- **Logical operations:**

```
1 !y # Negation: FALSE TRUE FALSE
2 x >= 2 | y < 3 # OR operation: FALSE TRUE TRUE
3 x < 2 & y >= 3 # AND operation: TRUE FALSE FALSE
4
5 x <- 1:100
6 sum(x > 50) # Q: What will R return and why?
```


Functions in R are Vectorized

- Suppose we have a function `f()` that we wish to apply to all elements of a vector `x`.
 - In many cases, we can accomplish this by simply calling `f()` on `x` itself.
- Simply put: **functions are vectorized!** meaning they are applied on each element of the input vector directly.
- **Examples:**

```
1 > sqrt(1:3)                # Square root: 1.000000 1.414214 1.732051
2 > round(c(1.2, 3.9, 0.4)) # Round up/down: 1 4 0
3
4 > y <- c(12, 5, 13)
5 > y + 4                    # Addition: 16 9 7
6 > for (i in 1:length(y)) { # If fn.s were NOT vectorized.. -> loop
7   print(y[i] + 4)
8 }
9
10 > '+'(y, 4)               # Addition..?
```

↪ The operator `+` is really a **function**!

Recycling Vectors in R

- **Recycling Rule:** When vectors of unequal length are involved in an operation, the shorter vector is recycled to match the length of the longer one.
 - `c(1,2,4) + c(6,0,9)` is valid.
 - `c(1,2,4) + c(6,0,9,20,22)` generates a warning.
- **Warning Case:** You get a warning if the length of the longer vector is not a multiple of the shorter one.
- **Example:**

```
1 > c(1,2,4) + c(6,0,9,20,22)
2 [1] 7 2 13 21 24
3
4 Warning message:
5 In c(1, 2, 4) + c(6, 0, 9, 20, 22) :
6   longer object length is not a multiple of shorter object length
```

Vector Indexing

- **Different Indexing Techniques:**

```
1 > y <- c(1.2, 3.9, 0.4, 0.12)
2 > y[1]      # Single indexing
3 [1] 1.2
4 > y[2:3]    # Range indexing
5 [1] 3.9 0.4
```

↪ Single indexing retrieves one element while range indexing retrieves a sequence.

```
1 > v <- 3:4 # Variable indexing
2 > y[v]
3 [1] 0.4 0.12
4 > z <- c(5, 12, 13)
5 > z[-1]    # Excluding elements
6 [1] 12 13
```

↪ Variable indexing allows dynamic element retrieval and indexing with negation operator `z[-1]` removes elements.

Filtering Vectors in R

- **Indexing vs. Filtering:**

- Filtering allows you to access, change, or remove elements **based on conditions rather than fixed index positions**.

```
1 > z <- c(5,2,-3,8)
2 > z[c(TRUE, FALSE, TRUE, TRUE)] # Logical filtering
3 [1] 5 -3 8
4 > which(z*z > 8) # Identifies indices
5 [1] 1 3 4
6
7 # Is z[which(z*z > 8)] filtering or indexing?
```

↪ Logical filtering: directly use logical vectors.

```
1 > w <- z[z*z > 8] # Conditional filtering
2 > w
3 [1] 5 -3 8
4 > x <- c(1,3,8,2)
5 > x[x > 3] <- 0; x # Replacing via conditions
6 [1] 1 3 0 2
```

↪ Conditional filtering: use conditions that generate logical vectors.

Vectors: Mixed Element Types

- If we create a vector with mixed elements (e.g., character and numeric), the resulting vector will be a character vector.
 - **Take away:** Vectors must have *homogeneous* elements.
 - ↪ If not? R converts *less flexible* to *more flexible type*.

```
1 > num.char.vec <- c(1, 3, "five", 7)
```

- Combining vectors results in a new, larger vector. If any element is a character, the entire combined vector becomes a character vector.
 - Caveat: Be cautious of *type coercion*.

```
1 > num.vec <- c(1, 3, 5, 7)
2 > comb.vec <- c(num.vec, num.char.vec)
```

Vectors: Type Conversion

- **Explicit Conversion Functions:**

- Functions like 'as.character' and 'as.numeric' enable explicit type conversion.

```
1 > numbers.vec <- c(-3, -2, -1, 0, 1, 2, 3)
2 > num2char <- as.character(numbers.vec)
3
4 > as.logical(numbers.vec) # Q: What will happen?
```

- **Be Cautious of 'NA':**

- Incompatible conversions will result in 'NA'.
↪ Best Practice? **Always validate the results after conversion.**

```
1 > char.vec <- c("1", "3", "five", "7")
2 > char2num <- as.numeric(char.vec); char2num # Print char2num
3 [1] 1 3 NA 7
```

Matrices: Two-Dimensional Vectors

Introduction to Matrices in R

- **Definition:** A matrix is a two-dimensional vector with uniform data types.
 - **Key Point:** Homogeneous elements (all of the same type).
- **Attributes:** Defined by the number of rows and columns.
 - Indexing begins from the top-left corner (e.g., `a[1,1]` indicates first row, first column).
- **Internal storage (i.e., 'how' data is stored in memory):**
 - Data stored in **column-major order**.

```
1 > input_data <- 1:9
2 > matrix(input_data, nrow = 3)
3      [,1] [,2] [,3]
4 [1,]    1    4    7
5 [2,]    2    5    8
6 [3,]    3    6    9
7
8 # Can we store data in row-major order? -> set byrow = TRUE.
```


Generating and Indexing Matrices

- Creating matrices with `matrix()`
 - Define dimensions using `nrow` and `ncol`, or let R decide.

```
1 > y <- matrix(c(1,2,3,4), nrow = 2, ncol = 2); y
2      [,1] [,2]
3 [1,]    1    3
4 [2,]    2    4
5
6 > z <- matrix(c(1,2,3,4), nrow = 2, byrow = T); z # Given nrow=2 & input vec. of length 4: ncol=?
7      [,1] [,2]
8 [1,]    1    2
9 [2,]    3    4
10
11 > matrix(c(1,2,3,4), nrow = 3) # Refer to this example for the question below!
```

⇒ Q: What happens when dimensions don't align? and why?

- Accessing matrix elements using the `[ ]` operator: specify row and column indices to retrieve values or sub-matrices.

```
1 > z[1,2] # Accessing a single element: row 1, column 2
2 > z[1,]  # Accessing all elements in row 1
3 > z[,2]  # Accessing all elements in column 2
```

More on Indexing Matrices

- Selecting multiple rows or columns using sequences:

```
1 > y <- matrix(1:9, nrow = 3); y
2      [,1] [,2] [,3]
3 [1,]    1    4    7
4 [2,]    2    5    8
5 [3,]    3    6    9
6
7 > y[2:3,] # Selecting rows 2 to 3.
8      [,1] [,2] [,3]
9 [1,]    2    5    8
10 [2,]    3    6    9
```

⇒ Q: How can we select columns 1 and 3? Hint: use `c()`

Indexing Matrices to Update Values

- **Example 1:**

```
1 > y[c(1,3),] <- matrix(rep(-1,6), nrow = 2); y
2      [,1] [,2] [,3]
3 [1,]  -1  -1  -1
4 [2,]   2   5   8
5 [3,]  -1  -1  -1
```

- **Example 2:**

```
1 > x <- matrix(nrow = 3, ncol = 3) # Creates an empty 3X3 matrix with NAs
2 > z <- matrix(c(4,5,2,3), nrow=2)
3
4 > x[2:3, 2:3] <- z; x
5      [,1] [,2] [,3]
6 [1,]  NA   NA   NA
7 [2,]  NA    4    2
8 [3,]  NA    5    3
```

Matrix: Dimension Reduction

- **Issue:** Indexing a row/column from a matrix often reduces it to a 1D vector.

```
1 > z <- matrix(1:8, nrow=4); z
2      [,1] [,2]
3 [1,]    1    5
4 [2,]    2    6
5 [3,]    3    7
6 [4,]    4    8
7
8 > z[2,]
9 [1] 2 6
```

1. The `attributes(z)` would show a dimension of 4x2.
2. But, `attributes(z[2,])` returns **NULL**, indicating it lacks additional information (metadata, e.g., dimension, names).

- **Solution:** Use `drop=FALSE` to retain the 2D matrix structure.

```
1 > r <- z[2,, drop = F]; r
2      [,1] [,2]
3 [1,]    2    6
4
5 > as.matrix(z[2,]) # Recall: Internal storage = column-major order. Q: What is R going to do?
6 > t(as.matrix(z[2,])) # Transpose it back to row vector.
```

Filtering Matrices with Conditions

- Filtering matrices enables you to access, modify, or eliminate elements **according to specified conditions** instead of fixed index positions.

```
1 > x <- matrix(c(1,2,3,2,3,4), nrow = 3)
2 > x[x[,2] >= 3,] # HERE
3      [,1] [,2]
4 [1,]  2    3
5 [2,]  3    4
```

- What is happening **HERE**?
 - First creates a logical vec. by comparing the values in 2nd column to 3.
 - Then use this vec. to select the 2nd and 3rd rows from matrix x.
- More advanced filtering examples:

```
1 > z <- c(5,12,13)
2 > x[z %% 2 == 1,]
3
4 > m <- matrix(1:6, nrow = 3); m
5 > m[m[,1] > 1 & m[,2] > 5,]
```

- Voluntary:** explain what is happening in words and share on Canvas!

Binding Elements to Matrices

- Create a vector & a matrix to bind together:

```
1 > one <- rep(1,4); one # Creates a vector 'one' that repeats the number 1 four times.
2 [1] 1 1 1 1
3
4 > z <- matrix(nrow = 4, ncol = 3) # Creates an empty 4X3 matrix which we can fill in by indexing.
5 > z[,1] <- 1:4 # Assign values to each column of z using the assignment operator.
6 > z[,2] <- c(1,1,0,0)
7 > z[,3] <- c(1,0,1,0)
8 > z
9      [,1] [,2] [,3]
10 [1,]    1    1    1
11 [2,]    2    1    0
12 [3,]    3    0    1
13 [4,]    4    0    0
```

- Binding elements using `cbind()` and `rbind()`:

```
1 > combMatrix <- cbind(one, z) # Caveat: Other memory-efficient binding methods are available!
2      one
3 [1,]    1    1    1    1
4 [2,]    1    2    1    0
5 [3,]    1    3    0    1
6 [4,]    1    4    0    0
7
8 > rbind(1, z, z) # Row binding. Q: What will happen?
9 > cbind(c(1,2), c(3,4)) # Column binding. Think about the size and content of resulting matrix.
```

Working with Row/Column Names

- **Recall:**

- We previously created a combined matrix, `combMatrix` which was missing numeric column indicators (e.g., `[,1]`, `[,2]`).

↪ Matrices in R, by default, don't have column or row names, but we can assign and retrieve them using `colnames()` and `rownames()`.

- **Checking Column Names:**

```
1 > colnames(combMatrix)
2 [1] "one" "" ""
```

- **Assigning Column Names:**

```
1 > colnames(combMatrix) <- c("one", "a", "b")
2 > combMatrix
3      one a b
4 [1,]  1 1 1
5 [2,]  1 2 0
6 [3,]  1 3 1
7 [4,]  1 4 0
```

Why Name Rows/Columns?

- **Ease of Data Access!:**

- Instead of remembering column positions, we can directly use column names to locate submatrices.

↪ Especially beneficial when working with large datasets with numerous columns/rows.

- **Example:**

```
1 > combMatrix[, "a"]  
2 [1] 1 2 3 4
```


Basic Arithmetic Operations on Matrices

- **Generate a 2X2 matrix y:**

```
1 > y <- matrix(1:4, nrow=2)
2 > y
3      [,1] [,2]
4 [1,]    1    3
5 [2,]    2    4
```

- Functions are vectorized for matrices as well!

```
1 > y * y      # Multiplication is done element-wise
2      [,1] [,2]
3 [1,]    1    9
4 [2,]    4   16
5
6 > 3 * y      # Scalar multiplication
7      [,1] [,2]
8 [1,]    3    9
9 [2,]    6   12
```

- For the ordinary matrix multiplications w/ dot products:

```
1 > y %*% y      # Use the %*% operator.
2      [,1] [,2]
3 [1,]    7   15
4 [2,]   10   22
```

Matrix: Computing Row- & Column- Means/Sums

- **Matrix Generation:**

```
1 > y <- matrix(seq(from=2, to=12, by=2), nrow=2)
2   [,1] [,2] [,3]
3 [1,]  2   6  10
4 [2,]  4   8  12
```

- **Row and Column-wise Computations:**

- Functions like `colMeans()` and `rowSums()` are specialized for row/column-wise operations.
- Using these functions eliminates manual loops, making computations faster and the code cleaner.
- **Example:**

```
1 > rowMeans(y) # row-wise mean of y
2 [1] 6 8
```

- **Total Matrix Computations ('Global' Statistics):**

- `mean()` considers the entire matrix as one vector (i.e., `mean(y) = 7`).

Arrays: Multi-Dimensional Vectors

Arrays in R: Introduction

- **Defining Arrays in R:**

- An array is a multi-dimensional vector.
- All elements within an array are of the *same data type*.

- **Creating an Array:**

- Envision arrays as **stacks of matrices**.
- Let's explore with an example:

```
1 > firstTest <- matrix(c(46,21,50,30,25,50), nrow = 2) # Start by defining matrices.
2 > secondTest <- matrix(c(50,30,25,21,50,46), nrow = 2)
```

⇒ Now, form the array:

```
1 > tests <- array(c(firstTest, secondTest), dim = c(2,3,2)); tests
2 , , 1
3
4      [,1] [,2] [,3]
5 [1,]  46   50   25
6 [2,]  21   30   50
7
8 , , 2
9
10     [,1] [,2] [,3]
11 [1,]  50   25   50
12 [2,]  30   21   46
```

Arrays in R: Access and Use

- **Accessing Elements in Arrays:**

- Square brackets are used: `array[row, column, outer dimension]`.
- For instance, `tests[1,2,1]` accesses the element in the first row, second column of the first matrix in our array `tests`.

- **Examples:**

```
1 > tests[1,2,1]      # Access a specific element.
2 [1] 50
3
4 > tests[1,,1]       # Retrieve all columns of the first row from the first matrix.
5 [1] 46 50 25
6
7 > tests[1,,2]       # Retrieve all columns of the first row from the second matrix.
8 [1] 50 25 50
```

- **Practical Use***:**

- Arrays provide a way to store multi-dimensional data. However, they are constrained to hold data of the same type.
 - In practice, Lists and Data Frames are more versatile and commonly used for multi-dimensional data structures. ~> Next week!

[First glance at] visualization for exploratory analysis

Midwest Data

```
> ggplot2::midwest ## Install the package first using install.package() if you haven't already!

# A tibble: 437 × 28
  PID county state area poptotal popdensity popwhite popblack popamerindian popasian popother
  <int> <chr> <chr> <dbl> <int> <dbl> <int> <int> <int> <int> <int>
1 561 ADAMS IL 0.052 66090 1271. 63917 1702 98 249 124
2 562 ALEXAN IL 0.014 10626 759 7054 3496 19 48 9
3 563 BOND IL 0.022 14991 681. 14477 429 35 16 34
4 564 BOONE IL 0.017 30806 1812. 29344 127 46 150 1139
5 565 BROWN IL 0.018 5836 324. 5264 547 14 5 6
6 566 BUREAU IL 0.05 35688 714. 35157 50 65 195 221
7 567 CALHOUN IL 0.017 5322 313. 5298 1 8 15 0
8 568 CARROLL IL 0.027 16805 622. 16519 111 30 61 84
9 569 CASS IL 0.024 13437 560. 13384 16 8 23 6
10 570 CHAMPA IL 0.058 173025 2983. 146506 16559 331 8033 1596
# 427 more rows
# 17 more variables: percwhite <dbl>, percblack <dbl>, percamerindian <dbl>, percasian <dbl>,
# percother <dbl>, popadults <int>, perchsds <dbl>, percollege <dbl>, percprof <dbl>,
# poppovertyknown <int>, percpovertyknown <dbl>, percbelowpoverty <dbl>,
# percchilbelowpovert <dbl>, percadultpoverty <dbl>, percelderlypoverty <dbl>, inmetro <int>,
# category <chr>
# Use `print(n = ...)` to see more rows
```

Building Up a Graph in Pieces

- Create a ggplot object and direct it to the correct data:

```
1 > p <- ggplot(data = midwest)
```

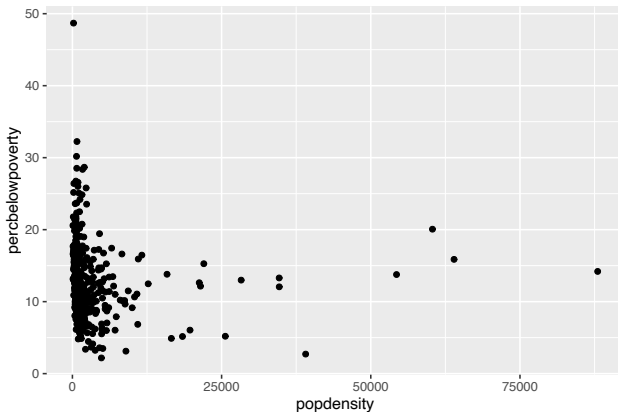
- **Mapping:** tell ggplot what visual aesthetics correspond to which variables:

```
1 > p <- ggplot(data = midwest,  
2           mapping = aes(x = popdensity,  
3                         y = percbelowpoverty))
```

- Other aesthetic mapping parameters: color, shape, size, etc.

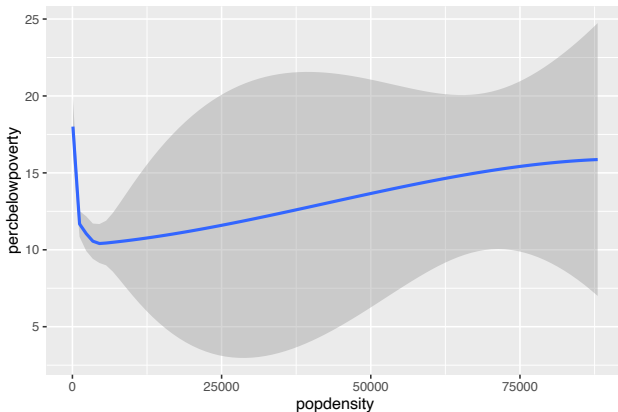
Adding a geom Layer

```
1 > ggplot(data = midwest,  
2         mapping = aes(x = popdensity,  
3                       y = percbelowpoverty)) +  
4     geom_point()
```



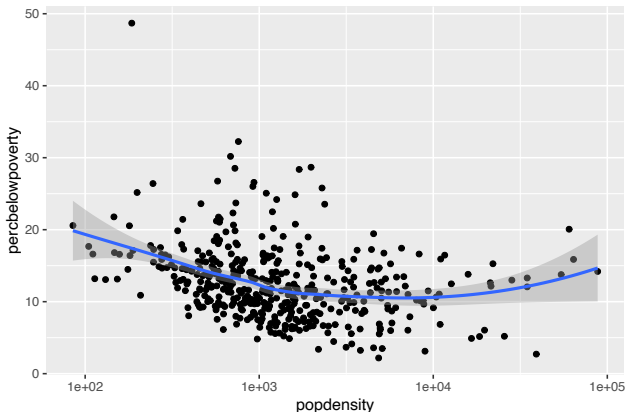
Trying a New geom

```
1 > ggplot(data = midwest,  
2         mapping = aes(x = popdensity,  
3                       y = percbelowpoverty)) +  
4   geom_smooth()
```



Layering geom is Additive!

```
1 > ggplot(data = midwest,  
2         mapping = aes(x = popdensity,  
3                       y = percbelowpoverty)) +  
4   geom_point() +  
5   geom_smooth() +  
6   scale_x_log10() # Scaling/normalizing to adjust for skewness & visual clarity
```

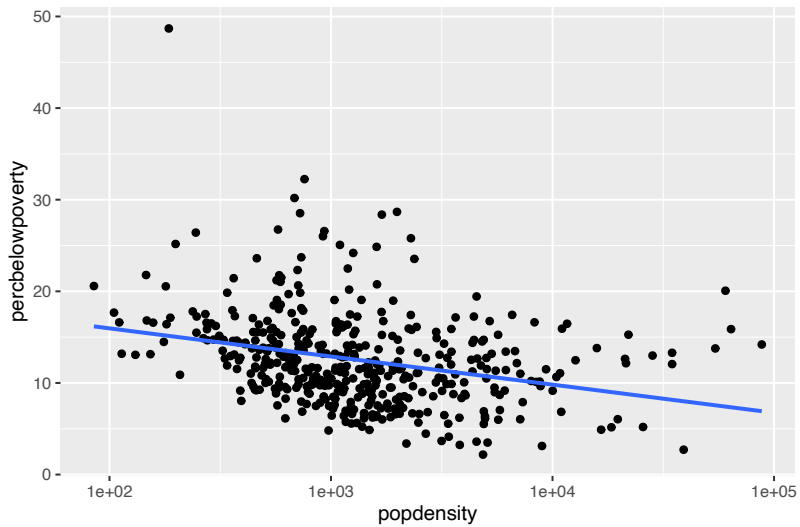


Geoms are Functions

- Geoms can take input arguments:

```
1 > ggplot(data = midwest,  
2         mapping = aes(x = popdensity,  
3                       y = percbelowpoverty)) +  
4   geom_point() +  
5   geom_smooth(method = "lm", se = F) +  
6   scale_x_log10()
```

↪ Tells `geom_smooth()` to do a linear fit with no error region.

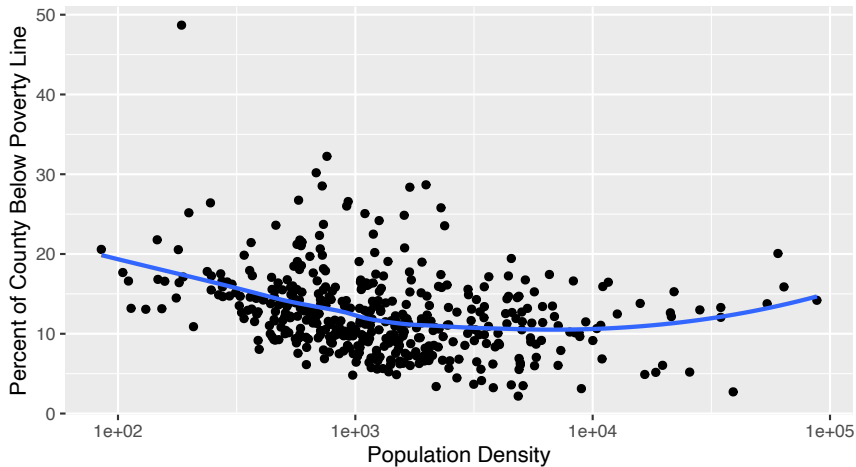


Adding Informative Labels

```
1 > ggplot(data = midwest,  
2         mapping = aes(x = popdensity,  
3                       y = percbelowpoverty)) +  
4   geom_point() +  
5   geom_smooth(method = "loess", se = F) +  
6   scale_x_log10() +  
7   labs(x = "Population Density",  
8        y = "Percent of County Below Poverty Line",  
9        title = "Poverty and Population Density",  
10       subtitle = "Among Counties in the Midwest",  
11       caption = "Source: US Census, 2000")
```

Poverty and Population Density

Among Counties in the Midwest

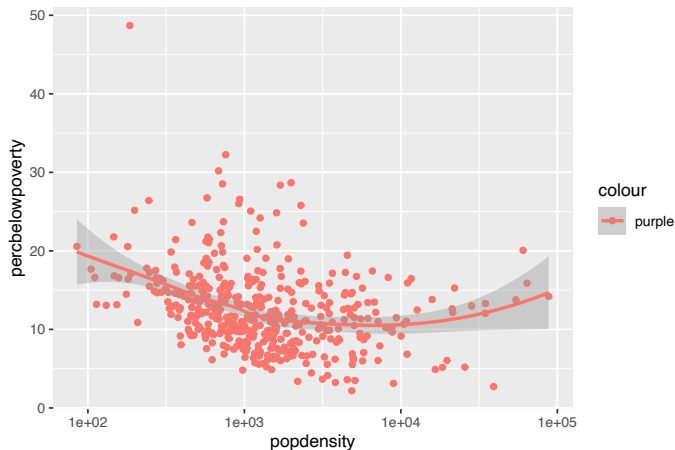


Source: US Census, 2000

Mapping vs. Setting Aesthetics

```
1 > ggplot(data = midwest,  
2         mapping = aes(x = popdensity,  
3                       y = percbelowpoverty,  
4                       color = "purple")) +  
5     geom_point() +  
6     geom_smooth() +  
7     scale_x_log10()
```


Wait, What Happened?

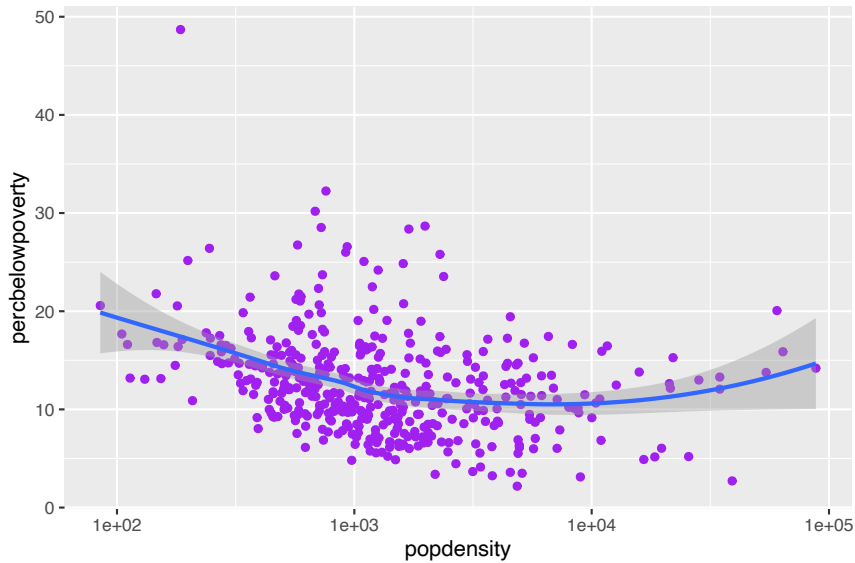


- Mapping always refers to variables!
 ↪ If passed a value other than a variable name, ggplot will implicitly create a variable with that value (in this case “purple” that is constant.)

Setting Aesthetics

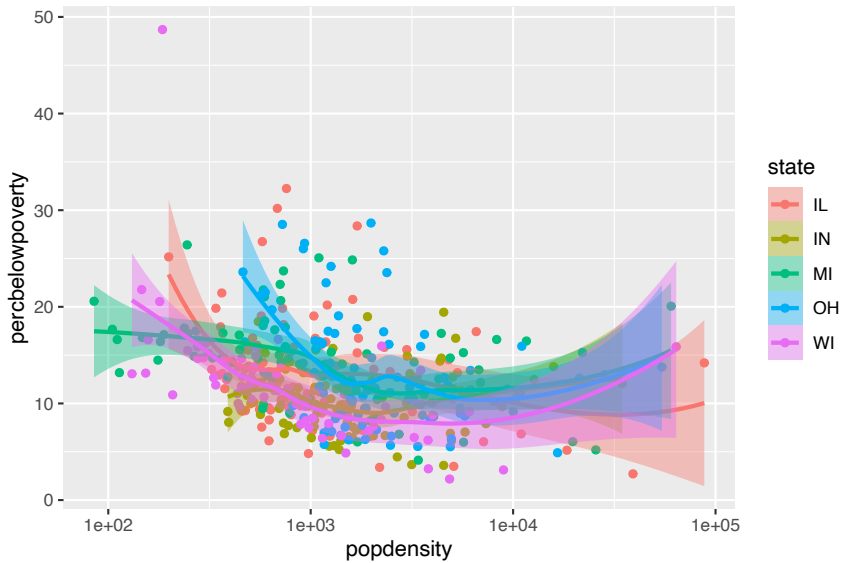
- Set the color outside the mapping (i.e., `aes()`) format:

```
1 > ggplot(data = midwest,  
2         mapping = aes(x = popdensity,  
3                       y = percbelowpoverty)) +  
4   geom_point(color = "purple") +  
5   geom_smooth() +  
6   scale_x_log10()
```



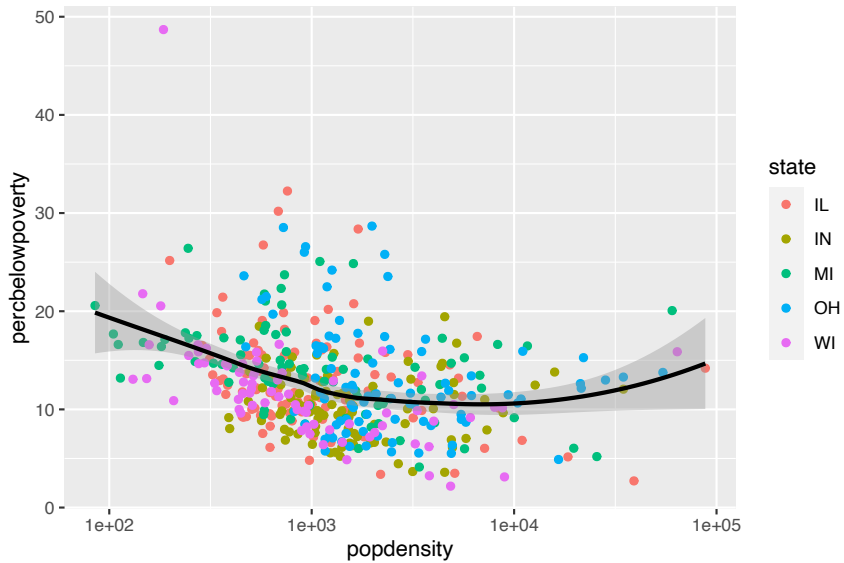
Mapping More Aesthetics

```
1 > ggplot(data = midwest,  
2         mapping = aes(x = popdensity,  
3                       y = percbelowpoverty,  
4                       color = state,  
5                       fill = state)) +  
6   geom_point() +  
7   geom_smooth() +  
8   scale_x_log10()
```



Mapping Can be Done on a Per Geom Basis

```
1 > ggplot(data = midwest,  
2         mapping = aes(x = popdensity,  
3                       y = percbelowpoverty)) +  
4   geom_point(mapping = aes(color = state)) +  
5   geom_smooth(color = "black") +  
6   scale_x_log10()
```



Have a Great Weekend! :)

Contact Information:

jaewon.yoo@iss.nthu.edu.tw

<https://j1yoo.github.io/>



Additional Materials for Review

I/O: Keyboard & Screen

➤ `z <- scan()`

`1: 12 5`

`3: 2`

`4:`

➤ `Read 3 items`

➤ `z`

`[1] 12 5 2`

➤ `w <- readline()` # input a line from the keyboard as a string
`abc de f`

➤ `w`

`[1] "abc de f"`

➤ `print(x)` # use this function in batch mode

Vectors: **any()**, **all()** function

- `x <- 1:10`
- `any(x > 8)`
[1] TRUE
- `any(x > 88)`
[1] FALSE
- `all(x > 88)`
[1] FALSE
- `if (all(x > 0)) print("yes")`
[1] "yes"

Vectors: NA & NULL

- NULL represents the null object in R
- NA represents 'Not Available' / Missing Values

➤ `x <- c(88, NA, 12, 168, 13)`

➤ `x`

`[1] 88 NA 12 168 13`

➤ `is.na(x)`

`[1] FALSE TRUE FALSE FALSE FALSE`

➤ `mean(x)`

`[1] NA`

➤ `mean(x, na.rm=T)`

`[1] 70.25`

➤ `x <- c(88, NULL, 12, 168, 13)`

➤ `mean(x)`

`[1] 70.25`

Vectors: `ifelse()` function

- `ifelse(b,u,v)`
 - where `b` is a boolean vector, and `u` and `v` are vectors.
 - The return value is a vector, element `i` of which is `u[i]` if `b[i]` is true, or `v[i]` if `b[i]` is false.

➤ `x <- 1:10`

➤ `y <- ifelse(x %% 2 == 0, 5, 12)`

➤ `y`
[1] 12 5 12 5 12 5 12 5 12 5

➤ `x <- c(5, 2, 9, 12)`

➤ `ifelse(x > 6, 2*x, 3*x)`
[1] 15 6 18 24

➤ `z <- 10`

➤ `ifelse(z>5, ifelse(z<8, 1, 2), 3) # nested ifelse()`
[1] 2

Vectors: Math. Functions

Functions	Description	Usage Example
abs(x)	Absolute value of x.	> abs(-2) [1] 2
sqrt(x)	Square root of x.	> sqrt(25) [1] 5
ceiling(x)	Returns the largest integer that is smaller than or equal to x.	> ceiling(3.475) [1] 4
floor(x)	Returns the smallest integer that is greater than or equal to x.	> floor(3.475) [1] 3
trunc(x)	Truncates (i.e., cuts off) the decimal places of x.	> trunc(5.99) [1] 5
round(x, digits=n)	Rounds up x to a specific number of decimal places.	> round(3.475, 2) [1] 3.48
signif(x, digits=n)	Rounds x to a specified number of digits.	> signif(0.00347, 2) [1] 0.0035
sin(x), cos(x), tan(x)	Trigonometric functions.	> sin(1) [1] 0.841471
asin(x), acos(x), atan(x)	Inverse trigonometric functions.	> asin(.841471) [1] 1
log(x, base=n)	Logarithm of x to the base n.	> log(2, base = 2) [1] 1
log(x)	Natural logarithm of x.	> log(10) [1] 2.302585
log10(x)	Common logarithm of x.	log10(10) [1] 1
exp(x)	Exponential of x. (e^x)	> exp(2.302585) [1] 10

Vectors: Stat. Functions

Functions	Description	Usage Example
mean(x)	Arithmetic mean of vector x.	> mean(c(1, 2, 3, 4, 50)) [1] 12
median(x)	Median value of vector x.	> median(c(1, 2, 3, 4, 50)) [1] 3
range(x)	Range of vector x (min, max). Calculation of range: diff(range(x))	> range(c(1, 2, 3, 4, 50)) [1] 1 50
IQR(x)	Quartile deviation of vector x.	> IQR(c(1, 2, 3, 4, 50)) [1] 2
sd(x)	Standard deviation of vector x.	> sd(c(1, 2, 3, 4, 50)) [1] 21.27205
var(x)	Variance of vector x.	> var(c(1, 2, 3, 4, 50)) [1] 452.5
sum(x)	Sum of all values in vector x.	> sum(c(1, 2, 3, 4, 50)) [1] 60
diff(x, lag=n)	Difference between elements of vector x. $(x_{n+i} - x_i)$ Default value of lag is 1.	> diff(c(1, 2, 4, 7, 11)) [1] 1 2 3 4
min(x)	Minimum value of vector x.	> min(c(1, 2, 3, 4, 50)) [1] 1
max(x)	Maximum value of vector x.	> max(c(1, 2, 3, 4, 50)) [1] 50

Matrix: Functions

Operators & Functions	Description
<code>+ - * / ^</code>	Operates on each element of the matrix.
<code>A %*% B</code>	Multiplication of matrix A and matrix B.
<code>cbind(A, B, ...)</code>	Merges/binds a matrix or a vector by column.
<code>colMeans(A)</code>	A vector containing the averages of each column in matrix A.
<code>crossprod(A)</code>	$t(A) \% * \% A \ (A^T A)$
<code>crossprod(A, B)</code>	$t(A) \% * \% B \ (A^T B)$
<code>colSums(A)</code>	A vector containing the sum of each column in matrix A.
<code>diag(A)</code>	A vector containing the main diagonal elements of matrix A.
<code>diag(x)</code>	A diagonal matrix with vector x as the main diagonal elements.
<code>diag(k)</code>	k x k identity matrix.
<code>eigen(A)</code>	A list containing the eigenvalue and the eigenvector of matrix A.
<code>rbind(A, B, ...)</code>	Merges/binds a matrix or a vector by row.
<code>rowMeans(A)</code>	A vector containing the averages of each row in matrix A.
<code>rowSums(A)</code>	A vector containing the sum of each row in matrix A.
<code>solve(A)</code>	Inverse of matrix A.
<code>solve(A, b)</code>	Solution for a system of equation, $Ax=b$.
<code>t(A)</code>	Transpose of matrix A. (A^T)
<code>tcrossprod(A)</code>	$A \% * \% t(A)$
<code>tcrossprod(A, B)</code>	$A \% * \% t(B)$